

Human Tracking Kalman Filter Evaluation

Session Summary — Thesis Social Navigation Project

Goal

Build a logging and analysis pipeline to evaluate a Kalman filter tracking a simulated walking person in ROS 2 / Gazebo, using the topics `/predicted_person_positions` and `/person_ground_truth`, and validate its accuracy against ground truth.

Tools Built

Script	Purpose
<code>kf_prediction_logger.py</code>	Subscribes to KF predictions + ground truth, logs both to CSV (sim-time aware)
<code>plot_kf_log.py</code>	Plots trajectory + velocity; breaks lines across detection gaps
<code>compare_true_vs_filtered_speed.py</code>	Quantifies true vs. KF speed bias, with time-window and turnaround filtering
<code>evaluate_prediction_accuracy.py</code>	Compares 2s-ahead predictions against actual future ground truth, vs. a naive baseline
<code>plot_trajectory_offset_figure.py</code>	Thesis-ready trajectory figure with annotated positional offset

Bugs Found and Fixed

Both verified via standalone simulation before deployment.

1. Duplicate-timestamp bug — `human_kf_predictor.py`

The dt clamp used a strict inequality ($0.0 < dt_{\text{raw}}$), so simultaneous/duplicate-timestamp messages fell through to a fallback meant for large gaps, fabricating $dt=0.1$. This spuriously dragged the Kalman velocity estimate down (confirmed: $0.094 \rightarrow 0.057$ m/s on affected rows). **Fixed** with an early return when $dt_{\text{raw}} \leq 0$.

2. Speed-halving clamp bug — `move_person_gazebo.py` / `move_person_gazebo2.py`

The dt-clamp safety ceiling (0.5s) was below the timer's own normal period ($update_dt=1.0s$), so it silently halved the simulated person's effective walking speed on every normal tick — turning a configured 0.2 m/s into a measured 0.1 m/s. **Fixed** by raising the ceiling to $update_dt \times 3.0$, preserving genuine stall protection.

Key Findings

Velocity bias (raw KF state vs. true speed)

Configuration	True speed (m/s)	KF speed (m/s)	Bias	% bias
<code>update_dt = 1.0</code> , Leg 1	0.2008 ± 0.0016	0.1691 ± 0.0429	-0.0317	-15.8%
<code>update_dt = 1.0</code> , Leg 2	0.2002 ± 0.0020	0.1746 ± 0.0361	-0.0256	-12.8%

update_dt = 0.3	0.1996 ± 0.0025	0.1871 ± 0.0233	-0.0124	-6.2%
-----------------	-----------------	-----------------	---------	-------

Improved from -12.8% to -15.8% (update_dt=1.0) down to -6.2% (update_dt=0.3), after tuning the simulated person's update rate for smoother motion. update_dt=0.1 was tried first but proved too aggressive — timer jitter at that rate (load average ~25–28 against 16 real CPU cores) made ground truth itself jagged. 0.3 was the validated sweet spot.

Positional offset

KF-filtered position sits consistently ~9–14 cm from ground truth (magnitude varied somewhat by capture window — likely viewing-angle dependent). Most plausible cause: the YOLO depth-sampling logic deliberately samples a lower-torso point on the person's bounding box, not their geometric center.

Sawtooth velocity ripple

A persistent ripple in raw KF velocity, tied to the simulated person's discrete (non-continuous) position updates — reduced but not eliminated by lowering update_dt.

vx, vy vs. pred_x, pred_y

These are different things: vx, vy is the raw Kalman velocity state (logged for diagnostics only); pred_x, pred_y is the EMA-smoothed, horizon-extrapolated position actually consumed by downstream obstacle-avoidance nodes. Today's velocity-bias analysis characterized the former. evaluate_prediction_accuracy.py was built to test the latter but not yet run to completion.

Process Notes

A significant amount of today's friction was file-management rather than science: stale or duplicated script versions, corrupted CSVs from abrupt process kills, wrong working directories, and a CPU-core-count misread (2 vs. the actual 16 cores) that was never fully resolved. Going forward, explicit verification (grep, wc -l, direct file content checks) before trusting any new capture is recommended.

Open Items for Next Session

- 1 Re-run the full clean capture at update_dt=0.3 (was in progress when the session ended).
- 2 Run evaluate_prediction_accuracy.py for the first time — gives the practically relevant “prediction error vs. naive baseline” number.
- 3 Resolve whether the 9–14 cm offset is camera/TF calibration or depth-sampling location, ideally by comparing offset direction across different walking headings.
- 4 Finalize the trajectory-offset thesis figure using a full-run average rather than a single window, for a more stable reported number.